arXiv:2604.22238v2 [cs.RO] 7 Jun 2026

CodeGraphVLP: Code-as-Planner Meets Semantic-Graph State for Non-Markovian Vision-Language-Action Models

Khoa Vo[†,1], Sieu Tran[†,1], Taisei Hanyu[1], Yuki Ikebe[1], Duy Nguyen[2], Nghi D. Q. Bui[3], Minh Vu[4]
Anthony Gunderman[1], Chase Rainwater[1], Anh Nguyen[5], and Ngan Le[1]

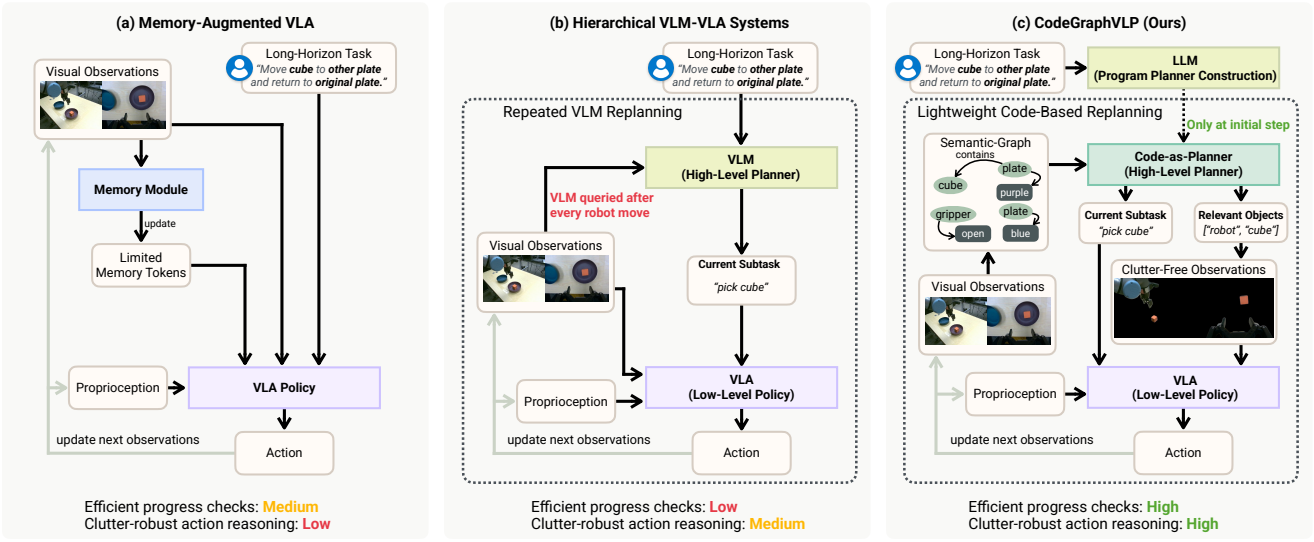

Fig. 1. **Architectures for non-Markovian long-horizon manipulation. (a) Memory-augmented VLA** equips a short-horizon policy with memory context, offering moderately efficient progress checks and limited robustness for action reasoning in clutter. **(b) Hierarchical VLM–VLA** uses a VLM planner to reason about subtasks and guide a VLA policy with subtask-level cues, improving clutter robustness but incurring highlatency due to repeated VLM inference online. **(c) CodeGraphVLP (Ours)** performs fast progress checks by synthesizing Code-as-Planner just once at initial step and executes it over a persistent semantic graph to produce clutter-free visual–language prompts to the VLA. Together, this enables efficient progress checks and reliable action reasoning in cluttered scenes.

***Abstract*— Vision-Language-Action (VLA) models promise generalist robot manipulation, but are typically trained and deployed as short-horizon policies that assume the latest observation is sufficient for action reasoning. This assumption breaks in non-Markovian long-horizon tasks, where task-relevant evidence can be occluded or appear only earlier in the trajectory, and where clutter and distractors make fine-grained visual grounding brittle. We present CodeGraphVLP, a hierarchical framework that enables reliable long-horizon manipulation by combining a persistent semantic-graph state with an executable code-based planner and progress-guided visual–language prompting. The semantic-graph maintains task-relevant entities and relations under partial observability. The synthesized planner executes over this semantic-graph to perform efficient progress checks and outputs a subtask instruction together with subtask-relevant objects. We use these outputs to construct clutter-suppressed observations that focus the VLA executor on critical evidence. On real-world non-Markovian tasks, CodeGraphVLP improves task completion over strong VLA baselines and history-enabled variants while substantially lowering planning latency compared to VLM-in-the-loop planning. We also conduct extensive ablation studies to confirm the contributions of each component.**

[†]Authors marked with this symbol contributed equally. [1]University of Arkansas, Fayetteville, AR, USA. [2]Max Planck Research School for Intelligent Systems and the University of Stuttgart, Stuttgart, Germany. [3]Center of AI Research, VinUniversity, VietNam. [4]TU Wien, Vienna, Austria. [5]University of Liverpool, Liverpool, U.K.

I. Introduction

Vision-Language-Action (VLA) models [1]–[9] have rapidly advanced generalist robot control by post-training Vision-Language Models (VLMs) backbone for action prediction on large-scale robotic manipulation data [10]–[12]. Most VLAs are trained and deployed as short-horizon policies that map the *latest* sensory input and instruction to the next action chunk, implicitly assuming that the current observation is sufficient for action reasoning. However, this assumption is fragile and limits real-world deployment, where long-horizon embodied tasks are intrinsically *non-Markovian*: the information required to choose the next action may not be contained in the latest observation due to partial observability, occlusions, and evolving task context. In other words, the correct next action can depend on task context and past observations.

Two strategies have emerged to extend VLAs to long-horizon tasks. *Memory-augmented VLAs* [13]–[16] (Figure 1(a)) support temporally referential grounding by integrating temporal context via step-wise histories, condensed memory tokens, or retrieval of task-relevant past experience. In parallel, *hierarchical VLM–VLA* [8], [17]–[20] (Figure 1(b)) place a VLM planner above a VLA executor,

repeatedly selecting a subtask and conditioning the VLA to execute it as a short closed-loop segment. This hierarchy is appealing because it reuses VLAs as generalist controllers while leveraging a general-purpose VLM for long-horizon reasoning and progress estimation.

Despite these advances, key non-Markovian challenges remain. Memory-augmented VLAs trade off capacity against efficiency, since scaling history (or retrieval) increases compute and latency, while bounded memory can miss the sparse past evidence needed for correct action selection. Hierarchical VLM–VLA systems reduce the burden on the executor by letting a VLM carry long-horizon reasoning and progress estimation, but they typically do not maintain a persistent grounded task state. Decisions therefore still rely heavily on the latest observation and language prompts, and the language-only interface can make the VLA brittle to clutter and distractors. Furthermore, keeping a VLM in the loop also adds substantial inference latency, undermining real-time manipulation feasibility. These limitations motivate a central question: *How to maintain a compact, explicit task state that preserves task-relevant evidence across time and supports reliable long-horizon control, while keeping inference practical for closed-loop robotics?*

To this end, we propose CodeGraphVLP (**CodeGraph** with **V**isual–**L**anguage **P**rompting) (Figure 1(c)), a hierarchical framework for non-Markovian long-horizon manipulation. Our approach integrates: (i) an executable **code-based planner** for efficient long-horizon progress estimation, (ii) a persistent **semantic graph state** that encodes task-relevant entities, attributes, and relations under partial observability, and (iii) **progress-guided visual–textual prompting** for attentive VLA execution in cluttered scenes.

CodeGraphVLP maintains a compact semantic-graph that persists across time and is updated online from observations to explicitly capture the evolving task context. Inspired by embodied programmatic control [21], we synthesize a task-specific code planner that queries this graph to check task progress through state-transition constraints and produce the next subtask instruction along with subtask-relevant objects. This programmatic reasoning provides a *lightweight mechanism for reasoning about task configurations* while avoiding repeated VLM calls for progress checks. We use these outputs to form progress-guided visual–textual prompts that suppress clutter and steer the VLA policy toward task-critical evidence for reliable action reasoning.

We evaluate CodeGraphVLP on three real-world tabletop manipulation tasks with non-Markovian dependencies, including one task that additionally emphasizes clutter-robust action reasoning. CodeGraphVLP achieves more reliable task completion than strong VLA baselines, memory-augmented variants, and VLM-in-the-loop alternatives, while substantially reducing planning latency compared to VLM-in-the-loop planning (Section IV-C). Overall, our key contributions are summarized as follows:

- We introduce CodeGraphVLP, which combines a code-based planner, a persistent semantic-graph that explicitly captures the scene, and progress-guided visual–language prompting to enable clutter-robust non-Markovian long-horizon manipulation.
- We show how executable code over the semantic graph can track progress and select subtasks efficiently while producing subtask-relevant entities for downstream execution.
- We design three real-world tabletop tasks with non-Markovian dependencies and clutter, and show that CodeGraphVLP improves task success while reducing planning latency compared to other history-enabled variants. Ablation studies further quantify the benefit of each component in the proposed CodeGraphVLP.

II. Related Work

A. Vision–Language–Action Models

Short-horizon VLAs. Vision–Language–Action (VLA) models [1], [3]–[7], [9] build generalist visuomotor policies by adapting pretrained vision–language backbones [22]–[26] on action prediction with large-scale robot demonstrations. Architecturally, autoregressive VLAs [5], [7] discretize actions into tokens and formulate control as next-token prediction, leveraging sequence modeling advances from language models, while *flow-based VLAs* [6], [8], [9] generate continuous trajectories through noise-to-action transformations (e.g., flow matching [27]), enabling smoother and higher-frequency control. Despite recent progress, most VLAs remain largely reactive, conditioning action generation on the latest observation while focusing primarily on low-level control. As a result, long-horizon task progress and persistent context are often left unmodeled, which becomes a bottleneck in non-Markovian settings where decisions depend on past observations that may be occluded or no longer visible.

To relax the short-horizon assumption, memory-augmented VLAs incorporate temporal context for history-dependent grounding, using mechanisms such as step-wise memories for temporally referential instructions [13], learned memory tokens [15], [16], or retrieval from external experience buffers [14]. However, these approaches typically maintain context as unstructured histories, bounded tokens, or retrieved snippets, limiting their ability to explicitly track long-horizon task progress.

In contrast, our method externalizes task context into a persistent semantic graph and an code-based planner that queries this structured state to track non-Markovian progress and generate grounded subtask instructions with relevant objects.

B. Non-Markovian Long-Horizon Reasoning

VLM-based Reasoning. A common approach to non-Markovian manipulation uses a pretrained VLM as a high-level reasoning module that converts visual observations and task descriptions into a global plan executed by a downstream controller. Methods in this category explore different prompting and feedback strategies to guide VLM reasoning, including visual prompt signals to highlight task-relevant regions [28], feedback from auxiliary modules such as scene description or success detection [29], and visual chain-of-thought reasoning that produces intermediate plans

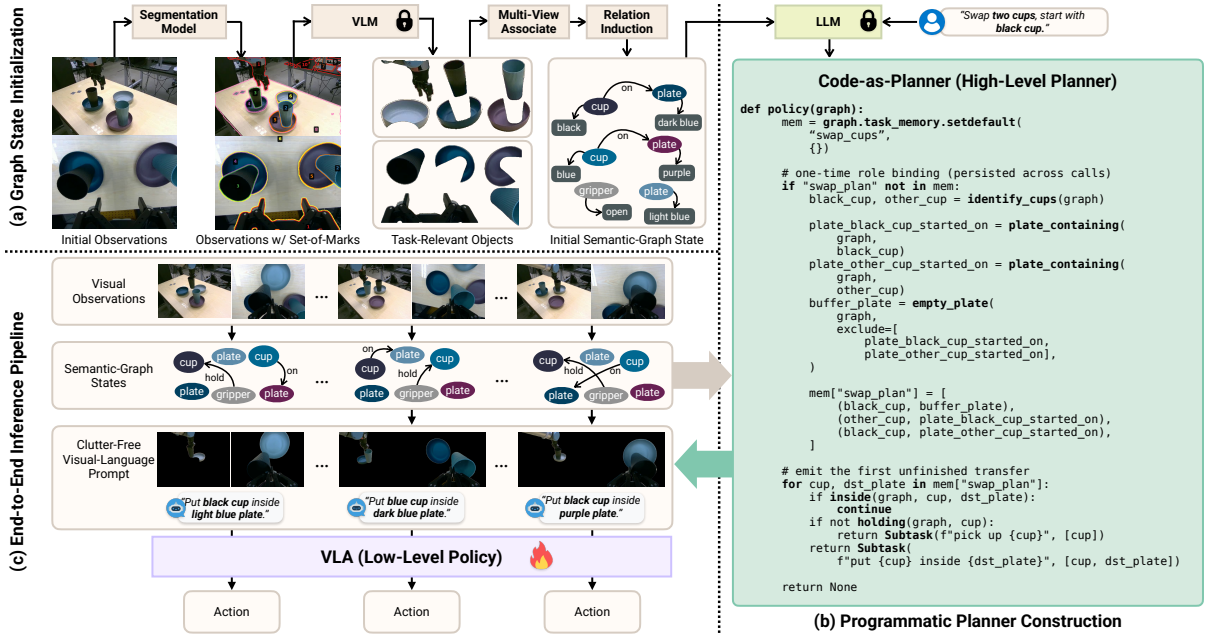

Fig. 2. Overview of CodeGraphVLP. **(a) Graph state initialization**: from the first multi-view observation, we segment objects, identify task-relevant entities, associate them across views, and induce relations to build a compact semantic graph. **(b) Programmatic planner construction**: given the task and a graph API, an LLM synthesizes an executable `policy(graph)` that performs progress checks over the graph and maintains lightweight task memory. **(c) End-to-end inference**: as new observations arrive, we update the semantic graph, execute the planner to produce the next subtask and relevant objects, construct clutter-free visual–language cues, and run a VLA policy for action prediction.

or rationales before action generation [30]. While effective for generating high-level task guidance, these approaches typically reason from a snapshot of the scene and may struggle to adapt when the environment changes or task progress becomes partially observable.

Another line of work performs *closed-loop VLM reasoning*, repeatedly querying the model after executing action chunks to update the plan from new observations. These approaches leverage structured representations and interactive feedback to enable iterative replanning under partial observability [8], [17], [31]. While this improves robustness in cluttered and dynamic environments, frequent VLM/LLM queries introduce significant computational overhead. Moreover, the planner typically communicates with the controller through language-only subtask instructions, leaving fine-grained visual grounding to the VLA, which can lead to mis-grounded actions in cluttered scenes.

We overcome such barriers by maintaining a persistent semantic graph state that is updated online and queried by an code-based planner to estimate task progress and generate subtasks, *without repeatedly invoking a VLM*. The planner also outputs *subtask-relevant objects*, enabling clutter-suppressing visual cues that guide the downstream VLA toward grounded action execution.

LLM-generated code for policy control. Recent work uses LLMs to synthesize executable programs that interface with perception and robot skills. [21] generates programs that compose APIs and control primitives to produce reactive policies from language instructions, while [32] generates constraint-based code that solves keypoint-level motion optimization over 3D observations. Rather than generating code to directly control actions or solve motion constraints, we formulate LLM-generated *code as a task-level planner operating over a persistent semantic-graph state* that is updated online. This structured and queryable state enables explicit reasoning about task progress and produces grounded subtasks, allowing the downstream VLA to focus on task-relevant entities and remain robust in cluttered scenes.

III. Method

A. Overview of CodeGraphVLP

Problem formulation. Formally, a VLA model is formulated under a Markovian assumption. At each timestep t, it conditions action generation on the latest observation and the task instruction, without any explicit mechanism to maintain a history-conditioned state. The robot observes a multi-view RGB observation $o_t = [I_t^1, \dots, I_t^n]$ together with proprioception s_t and receives a natural-language instruction l. Typically, a VLA uses a vision-language backbone to fuse (o_t, l) into a latent representation, followed by an action head that predicts a sequence of k future control actions (action chunking) [33], [34]:

$$\tau_t = (a_t, \dots, a_{t+k-1}) = \pi_\theta(o_t, s_t, l), \tag{1}$$

and re-query the model in closed loop as new observations arrive.

This Markovian assumption becomes unreliable in long-horizon tasks with non-Markovian dependencies, where the correct next action can depend on past evidence that may be occluded or absent from o_t. We capture this by defining the interaction history $h_t = (o_{0:t}, s_{0:t}, \tau_{0:t-1})$ and writing the optimal policy as

$$\tau_t = \pi^\star(h_t, l). \tag{2}$$

However, directly modeling h_t with longer observation windows, generic memory modules, or iterative VLM-in-the-loop planning is often impractical. Relevant evidence may be sparse or temporally distant and easily confounded by clutter, while memory and compute costs grow poorly with task horizon.

Overview of CodeGraphVLP. Our goal is to enable efficient long-horizon manipulation with VLAs under non-Markovian and cluttered conditions. We introduce CodeGraphVLP, which combines a Code-as-Planner with a persistent semantic-graph state that tracks task-relevant objects and relations over time. The code-based planner queries this structured state to estimate task progress and generate grounded subtasks without repeated VLM calls. We further design clutter-free visual–language prompting that exposes the VLA only to subtask-relevant context, improving visual grounding and reducing distractor-induced errors. An overview is shown in Figure 2.

B. Persistent Progress State via Semantic-Graph

At the core of our code planner is a persistent semantic-graph state $\mathcal{G}_t$ that summarizes task-relevant objects and relations, making scene context explicit and queryable. By filtering out irrelevant clutter, $\mathcal{G}_t$ remains compact under partial observability.

Semantic-graph state. We define $\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t)$, where $\mathcal{V}_t$ is the set of entity nodes and $\mathcal{E}_t$ is the set of typed relation edges. Each node $v_i \in \mathcal{V}_t$ represents an entity with a semantic name, per-view mask/box grounding, and task-relevant attributes. Each edge $e_{ij} = (v_i, v_j, r) \in \mathcal{E}_t$ connects $v_i, v_j \in \mathcal{V}_t$ with relation type r (e.g., *in*, *on*, and *near*).

Semantic-graph state initialization. Given the first observation o_0 and instruction l, we initialize $\mathcal{G}_0$ in four steps.

a) Object segmentation: For each view v, we use an off-the-shelf YOLOE [35] instance segmenter to extract a set of masks $\mathcal{M}^v = \{m_j^v\}_j$. We fine-tuned the segmenter with annotated robot arm masks from 10 videos in our teleoperated demonstrations to reliably segment its masks.

b) Identify task-relevant objects and their attributes: We overlay each segmented object with an outline and a numeric ID, following Set-of-Mark prompting [36], and query a VLM with the marked frames and instruction l to identify which objects are relevant to the task and their corresponding attributes. We then remove all irrelevant objects.

c) Multi-view association: Due to occlusion, each view may contain only a subset of task-relevant objects. We therefore link masks across views into view-consistent graph nodes. We first match objects by *semantic distance*: CLIP visual features [37] are extracted from each masked region, and we match cross-view pairs whose cosine distance is below a threshold τ_{vis}. Afterwards, we resolve the remaining cross-view associations using a *geometric distance* based on relative spatial layout. Let $\mathcal{A}$ denote the set of anchors (nodes already matched by semantic distance), and let $\mathbf{c}(m^v) = (x_m^v, y_m^v) \in \mathbb{R}^2$ be the center-of-mass of mask m^v of an unmatched object in view v. We compute distance of m^v to every anchor $a \in \mathcal{A}$ in the same view as follows:

$$d(m^v, a^v) = \|\mathbf{c}(m^v) - \mathbf{c}(a^v)\|_2 \tag{3}$$

forming a distance signature $\mathbf{d}^v(m) = [d(m^v, a^v)]_{a\in\mathcal{A}}$. We further normalize it by the maximum anchor distance in the view, $\tilde{\mathbf{d}}^v(m) = \mathbf{d}^v(m) / \max_{a\in\mathcal{A}} d(m^v, a^v)$, to account for scale differences between views. Finally, we match the remaining objects across views by comparing their normalized signatures $\tilde{\mathbf{d}}$ and accepting nearest-neighbor matches under a threshold τ_{geo}.

d) Relation induction: With view-consistent nodes in place, relation edges $\mathcal{E}_0$ are induced with lightweight geometric rules on the segmentation masks. Proximity (*near*) thresholds the normalized center-distance between objects, containment (*in*) checks whether one mask substantially encloses another, and support (*on*) uses consistent vertical ordering and boundary contact in image space.

Online semantic-graph state update. At each timestep $t > 0$, we track task-relevant objects from $\mathcal{G}_{t-1}$ to the new observation o_t using Cutie [38], which updates the per-view mask groundings of existing nodes. To recover from tracker drift and to discover new objects, we additionally run the instance segmenter on o_t and pass candidate masks through the same relevance filtering and multi-view association procedure as described in semantic-graph state initialization phase. When a candidate region corresponds to an existing node, we merge it as a correction; otherwise, we add it as a new node only if it is deemed task-relevant and then refresh edges using relation induction rules.

C. Code-as-Planner

Constructing programmatic planner. We represent the planner as an executable Python program $\mathcal{P}$ that is repeatedly invoked online during manipulation deployment. Given the current semantic-graph, it outputs the next subtask instruction and the subtask-relevant objects that should be grounded for action reasoning:

$$\left(l_t^{\text{sub}}, \mathcal{O}_t^{\text{rel}}\right) = \mathcal{P}(\mathcal{G}_t). \tag{4}$$

To instantiate $\mathcal{P}$ for a long-horizon instruction l, we follow programmatic embodied control [21], [29], [39] and prompt an LLM once at task initialization with l, the graph schema, and the initial graph $\mathcal{G}_0$. The LLM synthesizes a task-specific Python planner with three components. First, it generates helper routines that implement common graph queries (e.g., filtering objects by class and attributes, locating the container an object is currently *in*, checking whether the robot is *holding* an object, and selecting an empty buffer container). Second, it defines lightweight boolean predicates that encode task constraints and progress conditions over $\mathcal{G}_t$ (e.g., single-object holding, exclusivity constraints for containers, and relation checks for *in*/*on*/*holding*). Third, it implements a main `policy(graph)` function that uses these primitives to select the next subtask and returns $\left(l_t^{\text{sub}}, \mathcal{O}_t^{\text{rel}}\right)$.

We instruct the LLM to first identify task roles and ordering constraints from $\mathcal{G}_0$, and then implement a reactive program that grounds roles from semantic fields (class names, attributes, relations). Crucially, the code maintains a lightweight persistent list `graph.task_memory` that records which subtasks have already been completed, preventing redundant execution across repeated replanning calls. This one-time synthesis amortizes LLM/VLM inference cost and yields a planner that can be executed cheaply without keeping a large LLM/VLM in the loop.

Planner structure and task memory maintenance. During deployment, $\mathcal{P}$ is called after each executed action chunk with the latest graph $\mathcal{G}_t$. At each call, the synthesized `policy(graph)` queries $\mathcal{G}_t$ and uses simple graph predicates (e.g., *holding* and *in*) to check whether the current subtask has already been satisfied. If so, it appends the subtask to `graph.task_memory` and advances to the next unfinished subtask; otherwise, it re-issues the same instruction. It then returns $\left(l_t^{\text{sub}}, \mathcal{O}_t^{\text{rel}}\right)$, where $\mathcal{O}_t^{\text{rel}}$ contains only the objects needed to ground the immediate decision.

Example (swap cups). Consider swapping two cups, starting with the black cup (Figure 2(b)). The synthesized `policy(graph)` performs one-time role binding, identifying `black_cup`, `other_cup`, their start plates, and an empty `buffer_plate` for temporary storage, and caches a three-step `swap_plan` (respecting container exclusivity) in `graph.task_memory`. At runtime, it scans `swap_plan` and emits the first unfinished transfer, choosing between *pick up* and *put … inside …* based on *holding*/*in* predicates and returning only the subtask-relevant objects for grounding. We use $\mathcal{O}_t^{\text{rel}}$ to construct the clutter-free visual–textual cues for the VLA policy described next. Note that the planner only reads $\mathcal{G}_t$; graph construction and online updates are handled by Section III-B.

Overall, we alternate between updating $\mathcal{G}_t$, invoking $\mathcal{P}$, and executing one action chunk with the VLA, which keeps planning reactive while avoiding VLM-in-the-loop inference. Expressing planning as executable code over $\mathcal{G}_t$ enables efficient and interpretable long-horizon subtask reasoning, while the explicit $\mathcal{O}_t^{\text{rel}}$ provides grounded context that improves robustness under clutter.

D. Clutter-Free Visual-Language Prompting

Long-horizon tasks are often executed in cluttered workspaces, where many visible objects are irrelevant to the current decision and can distract a VLA from correct grounding. We therefore convert the planner output $\left(l_t^{\text{sub}}, \mathcal{O}_t^{\text{rel}}\right)$ together with the semantic-graph state $\mathcal{G}_t$ into a progress-guided visual–language prompt for the VLA executor.

Subtask language cue. Following hierarchical long-horizon systems that guide low-level execution with subtask-level language [17], [20], we condition the VLA executor on the planner-produced instruction l_t^{sub} instead of the long-horizon task description l. Keeping l_t^{sub} short and imperative focuses the language context on the immediate decision and aligns it with the subtask-relevant object set $\mathcal{O}_t^{\text{rel}}$, improving action reasoning in cluttered scenes.

Clutter-free visual cue. Subtask-level language reduces linguistic ambiguity, but the VLA executor still has to perform fine-grained visual grounding from raw RGB observations, which has been shown to be brittle under clutter and distractors [40]. We address this brittleness by constructing a clutter-free observation that retains only the subtask-relevant objects $\mathcal{O}_t^{\text{rel}}$. For each view v, let $m_{i,t}^v$ denote the binary segmentation mask of object i rendered in that view by $\mathcal{G}_t$. We define the view-specific retention mask as:

$$M_t^v = \max_{i \in \mathcal{O}_t^{\text{rel}}} m_{i,t}^v, \tag{5}$$

which keeps only the subtask-relevant objects. We then form a clutter-free image by blacking out all other pixels:

$$\tilde{I}_t^v = I_t^v \odot M_t^v, \tag{6}$$

and define the clutter-free multi-view observation as $\tilde{o}_t = [\tilde{I}_t^1, \ldots, \tilde{I}_t^n]$.

VLA execution. The executor VLA predicts the next action chunk from $(\tilde{o}_t, s_t, l_t^{\text{sub}})$, and we apply the same prompting during fine-tuning to match the training and deployment inputs. By exposing only subtask-critical visual evidence and suppressing distractors, the prompting improves clutter-robust action reasoning of downstream VLA policies.

Training objective. We fine-tune the VLA via imitation learning using training inputs that match the progress-guided executor inputs used at test time. For each teleoperated trajectory paired with a long-horizon instruction l, we first build the semantic-graph $\mathcal{G}_t$ from the recorded observations using our semantic-graph construction and online update pipeline. We then execute the synthesized code planner $\mathcal{P}(\mathcal{G}_t)$ at each timestep to produce $\left(l_t^{\text{sub}}, \mathcal{O}_t^{\text{rel}}\right)$, and mask the multi-view images to form the clutter-free observation $\tilde{o}_t$. This converts each long-horizon demonstration into a set of subtask-conditioned, object-focused training tuples. We optimize the VLA parameters θ on $\mathcal{D} = \{(\tilde{o}_i, s_i, \tau_i, l_i^{\text{sub}})\}_{i=1}^N$ via maximum-likelihood estimation:

$$\max_\theta \mathbb{E}_{(\tilde{o},s,\tau,l^{\text{sub}})\sim\mathcal{D}} \left[\log \pi_\theta(\tau|\tilde{o}, s, l^{\text{sub}})\right]. \tag{7}$$

IV. Experiments

To comprehensively evaluate CodeGraphVLP, we design our experiments to answer three questions:

Q1. Can CodeGraphVLP complete non-Markovian long-horizon tasks more reliably than strong VLA baselines, especially in cluttered scenes?

Q2. Does programmatic planning over a persistent semantic-graph enable more efficient progress estimation than history-enabled VLAs or VLM-in-the-loop hierarchies?

Q3. How much does progress-guided visual–textual prompting improve clutter-robust action reasoning during execution?

A. Experimental Setups

Tasks. We evaluate CodeGraphVLP on a suite of three real-world tabletop manipulation tasks that require reasoning over past observations and clutter-robust action reasoning,

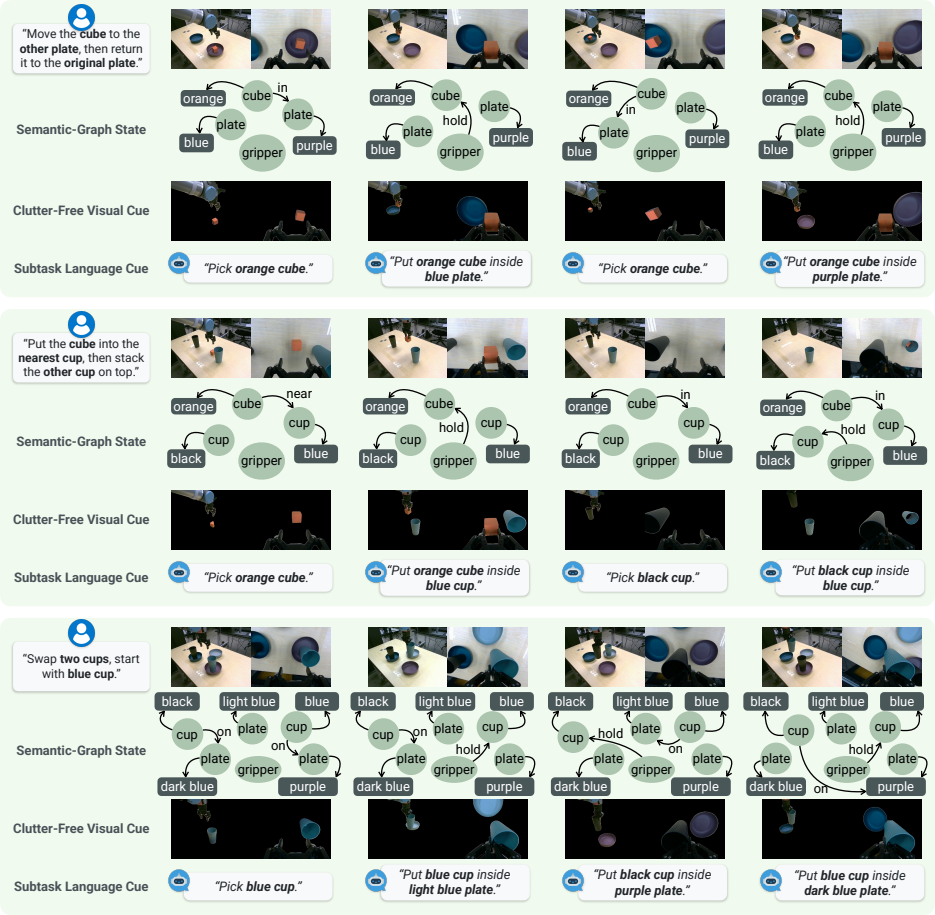

Fig. 3. **Qualitative rollouts of CodeGraphVLP on our three real-world tasks (Pick-and-Place Twice, Place-and-Stack, and Swap Cups).** For each task, we show multi-view RGB inputs with the overall instruction, the semantic-graph state $\mathcal{G}_t$, and the progress-guided prompts used by the VLA policy: clutter-free visual cues that retain only subtask-relevant objects and the planner-produced subtask language cues.

inspired by non-Markovian long-horizon benchmarks [15]:

(i) Pick-and-Place Twice. Two plates are placed on the tabletop, and a cube is initialized on a randomly chosen plate. The robot must pick the cube and place it onto the other plate, then pick it again and return it to the original plate. This task is history-dependent because both the initial and terminal configurations are visually similar, so a policy can prematurely predict that it is already done.

(ii) Place-and-Stack. Two cups are placed on opposite sides of the workspace, and a cube is initialized near one of the cups. The robot must first place the cube into the nearest cup, then pick up the other cup and stack it onto the cup that contains the cube. After the cube is inserted, it is completely occluded, so selecting the empty cup to stack depends on past observations rather than the latest RGB input alone.

(iii) Swap Cups. Three plates are placed on the tabletop, and two cups (black and blue) are randomly placed on two of them, leaving one plate empty as a temporary buffer. The goal is to swap the positions of the two cups, with the additional constraint that the policy must start by moving either the black cup or the blue cup (two task variants). This requires memorizing the initial cup–plate assignments and executing multi-step swaps without reverting completed steps.

Demonstrations. We collect teleoperated demonstrations for fine-tuning all VLA models: 100 for Pick-and-Place Twice, 100 for Place-and-Stack, and 200 for Swap Cups, with 100 episodes that start with the blue cup and 100 that start with the black cup.

Baselines. We compare CodeGraphVLP against strong open VLA baselines, including π_0 [6], π_0 FAST [7], $\pi_{0.5}$ [8], and Gr00T N1.5 [9]. These baselines operate directly on raw multi-view RGB observations without progress-guided prompting. To probe the benefit of raw temporal context, we also evaluate a history-enabled Gr00T N1.5 variant that receives four past frames sampled at 1 s intervals. We fine-tune and evaluate all baselines under the same protocol as CodeGraphVLP, using identical task prompts and replanning cadence.

Implementation details. We instantiate the VLA executor in CodeGraphVLP with π_0 [6] and fine-tune all VLA models (ours and baselines) from their public checkpoints for 50K iterations using low-rank adaptation [41] (learning rate 1×10^{-5}, batch size 128) on four NVIDIA A6000 GPUs. At test time, inference runs on a single GPU and we execute a cut-off horizon of $H = 10$ actions per forward pass before

TABLE I

COMPARISON OF METHODS ON REAL-WORLD NON-MARKOVIAN LONG-HORIZON TASKS. ALL REPORTED VALUES ARE SUCCESS RATES (%).

Method	Pick-and-Place Twice		Place-and-Stack		Swap Cups		Avg.
	PnP Once	Success Rate	Drop Cube	Success Rate	Stage Cup	Success Rate	
π_0 FAST [7]	0	0	0	0	0	0	0.0
π_0 [6]	0	0	60	40	55	50	30.0
$\pi_{0.5}$ [8]	5	0	35	5	25	10	5.0
Gr00T N1.5 [9]	50	35	40	40	70	20	31.7
Gr00T N1.5 + Multi-frame	100	75	50	50	90	45	56.7
CodeGraphVLP (ours)	**100**	**80**	**95**	**80**	**100**	**85**	**81.7**

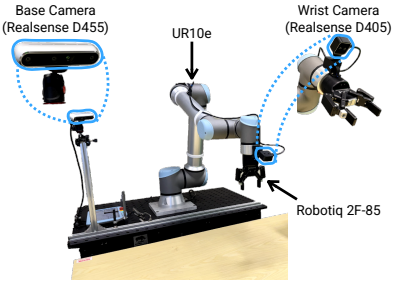

Fig. 4. Robot experimental setup on a UR10e manipulator with a parallel gripper. The system receives both a global scene observation and a wrist-view observation near the end-effector.

replanning. We use GPT-5 [42] to synthesize the executable planning program once at the beginning of each task.

Hardware setup. All experiments are conducted on the UR10e robotic arm with a Robotiq 2F-85 parallel-jaw gripper (Figure 4). We capture RGB observations from two synced camera streams, including a fixed camera placed in an over-shoulder viewpoint of the robot, and a wrist-mounted camera placed at the wrist–gripper interface. During teleoperation, we record demonstrations and log observations at 10 Hz.

B. Main Results

Table I reports success rates across three real-world non-Markovian tasks, including intermediate milestones that capture partial progress. CodeGraphVLP consistently achieves the highest performance, significantly outperforming all baselines, including the strongest history-augmented VLA. Improvements in both full-task and partial completion metrics suggest that CodeGraphVLP not only executes subtasks more reliably in clutter but also maintains more consistent long-horizon progress. Representative rollouts are shown in Figure 3.

Among the baselines, π_0 FAST fails on all tasks, suggesting that autoregressive action decoding may be brittle for long-horizon closed-loop control, where greedy predictions can collapse to inert behaviors under ambiguous observations. In contrast, VLAs with diffusion or flow-based decoders appear more stable, and conditioning on temporal context further improves performance. However, even the strongest history-augmented baseline remains substantially below CodeGraphVLP, indicating that unstructured temporal context alone is insufficient to resolve non-Markovian dependencies or clutter-induced grounding errors without explicit progress reasoning and structured visual cues.

TABLE II

ABLATION ON SUBTASK PLANNING EFFICIENCY ON SWAP CUPS.

Code-as-Planner	Semantic Graph	Success Rate	Latency (sec/step)
✗	✗	25	2.967
✗	✓	55	3.142
✓	✓	85	0.328

TABLE III

ABLATION ON CLUTTER-FREE VISUAL–LANGUAGE PROMPTING ON SWAP CUPS.

Method	Success rate
CodeGraphVLP w/o clutter-free visual	40
CodeGraphVLP	85

C. Ablation Analysis

Unless otherwise stated, we conduct all ablation studies on the Swap Cups task, which requires both non-Markovian reasoning and clutter-robust execution.

Efficiency benefits of Code-as-Planner. To evaluate the efficiency and reliability of Code-as-Planner, we compare it with two VLM-based planners that infer the next subtask online at each replanning step. Both use the latest multi-view RGB observation, while one additionally receives our semantic-graph progress state. As shown in Table II, the progress state improves VLM planning, but RGB-only reasoning remains insufficient in non-Markovian settings. Even so, VLM-in-the-loop planning remains slower and less reliable, whereas Code-as-Planner achieves higher success with substantially lower latency through one-time synthesis and lightweight graph queries.

How effective is visual–textual prompting for VLA execution? We isolate the effect of the clutter-free visual cue by removing the subtask-relevant object set while keeping the subtask language cue unchanged. Concretely, we train and deploy the VLA policy on the original multi-view RGB observations instead of masking to the subtask-relevant regions. As shown in Table III, this reduces success from 85% to 40%, indicating that progress-guided perceptual inputs are important for clutter-robust action reasoning.

V. Conclusion

We introduced CodeGraphVLP, a framework that enables long-horizon manipulation by combining structured scene memory with programmatic planning and grounded visual–language prompting. Experiments on real-world tasks show that this design improves task success while reducing planning latency compared to VLM-in-the-loop and memory-augmented VLA baselines.

Limitations. Our pipeline relies on foundation models for both semantic-graph construction and program synthesis, so performance depends on their capabilities and failure modes. In particular, the attributes and relations inferred by the VLM can be sensitive to camera viewpoints and visual quality, and the generated Code-as-Planner requires careful prompt design to ensure reliable, executable logic. Future work includes developing an open-world semantic-graph generation module that streamlines the construction of persistent semantic-graph state, and automated verification and self-checks for synthesized code-based planners.

References

[1] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn *et al.*, "Rt-1: Robotics transformer for real-world control at scale," in *arXiv preprint arXiv:2212.06817*, 2022.

[2] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia *et al.*, "Rt-2: Vision-language-action models transfer web knowledge to robotic control," in *CoRL*, 2023, pp. 2165–2183.

[3] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees *et al.*, "Octo: An open-source generalist robot policy," in *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, 2024.

[4] X. Li, M. Liu, H. Zhang, C. Yu, J. Xu, H. Wu *et al.*, "Vision-language foundation models as effective robot imitators," in *ICLR*, 2024.

[5] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair *et al.*, "OpenVLA: An open-source vision-language-action model," in *CoRL*, 2024.

[6] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn *et al.*, "π_0: A vision-language-action flow model for general robot control," *arXiv preprint arXiv:2410.24164*, 2024.

[7] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong *et al.*, "Fast: Efficient action tokenization for vision-language-action models," *arXiv preprint arXiv:2501.09747*, 2025.

[8] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess *et al.*, "$\pi_{0.5}$: a vision-language-action model with open-world generalization," *arXiv preprint arXiv:2504.16054*, 2025.

[9] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan *et al.*, "Gr00t n1: An open foundation model for generalist humanoid robots," *arXiv preprint arXiv:2503.14734*, 2025.

[10] H. R. Walke, K. Black, T. Z. Zhao, Q. Vuong, C. Zheng, P. Hansen-Estruch *et al.*, "Bridgedata v2: A dataset for robot learning at scale," in *CoRL*. PMLR, 2023, pp. 1723–1736.

[11] A. O'Neill, A. Rehman, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee *et al.*, "Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0," in *ICRA*. IEEE, 2024, pp. 6892–6903.

[12] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti *et al.*, "Droid: A large-scale in-the-wild robot manipulation dataset," *arXiv preprint arXiv:2403.12945*, 2024.

[13] H. Ahn, O. Kwon, K. Kim, J. Jeong, H. Jun, H. Lee *et al.*, "Visually grounding lanuage instruction for history-dependent manipulation," in *ICRA*, May 2022.

[14] A. Sridhar, J. Pan, S. Sharma, and C. Finn, "Memer: Scaling up memory for robot control via experience retrieval," 2025.

[15] M. Koo, D. Choi, T. Kim, K. Lee, C. Kim, Y. Seo *et al.*, "HAMLET: Switch your vision-language-action model into a history-aware policy," in *ICLR*, 2026.

[16] H. Shi, B. Xie, Y. Liu, L. Sun, F. Liu, T. Wang *et al.*, "MemoryVLA: Perceptual-cognitive memory in vision-language-action models for robotic manipulation," in *ICLR*, 2026.

[17] L. X. Shi, brian ichter, M. R. Equi, L. Ke, K. Pertsch, Q. Vuong *et al.*, "Hi robot: Open-ended instruction following with hierarchical vision-language-action models," in *ICML*, 2025.

[18] C.-P. Huang, Y.-H. Wu, M.-H. Chen, Y.-C. F. Wang, and F.-E. Yang, "Thinkact: Vision-language-action reasoning via reinforced visual latent planning," in *NeurIPS*, 2025.

[19] C.-P. Huang, Y. Man, Z. Yu, M.-H. Chen, J. Kautz, Y.-C. F. Wang *et al.*, "Fast-thinkact: Efficient vision-language-action reasoning via verbalizable latent planning," *arXiv preprint arXiv:2601.09708*, 2026.

[20] D. Li, Y. Zhang, M. Cao, D. Liu, W. Xie, T. Hui *et al.*, "Towards long-horizon vision-language-action system: Reasoning, acting and memory," in *ICCV*, 2025, pp. 6839–6848.

[21] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter *et al.*, "Code as policies: Language model programs for embodied control," in *ICRA*, 2023, pp. 9493–9500.

[22] J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson *et al.*, "Flamingo: a visual language model for few-shot learning," in *NeurIPS*, vol. 35, 2022.

[23] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter *et al.*, "Palm-e: an embodied multimodal language model," in *ICML*, ser. ICML'23. JMLR.org, 2023.

[24] X. Chen, J. Djolonga, P. Padlewski, B. Mustafa, S. Changpinyo, J. Wu *et al.*, "On scaling up a multilingual vision and language model," in *CVPR*, June 2024, pp. 14 432–14 444.

[25] S. Karamcheti, S. Nair, A. Balakrishna, P. Liang, T. Kollar, and D. Sadigh, "Prismatic vlms: Investigating the design space of visually-conditioned language models," in *ICML*, 2024.

[26] L. Beyer, A. Steiner, A. S. Pinto, A. Kolesnikov, X. Wang, D. Salz *et al.*, "Paligemma: A versatile 3b vlm for transfer," *arXiv preprint arXiv:2407.07726*, 2024.

[27] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *ICLR*, 2023.

[28] K. Fang, F. Liu, P. Abbeel, and S. Levine, "Moka: Open-world robotic manipulation through mark-based visual prompting," *Robotics: Science and Systems (RSS)*, 2024.

[29] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence *et al.*, "Inner monologue: Embodied reasoning through planning with language models," in *CoRL*, 2022.

[30] M. Zawalski, W. Chen, K. Pertsch, O. Mees, C. Finn, and S. Levine, "Robotic control via embodied chain-of-thought reasoning," *arXiv preprint arXiv:2407.08693*, 2024.

[31] K. Rana, J. Haviland, S. Garg, J. Abou-Chakra, I. Reid, and N. Suenderhauf, "Sayplan: Grounding large language models using 3d scene graphs for scalable task planning," in *CoRL*, 2023.

[32] W. Huang, C. Wang, Y. Li, R. Zhang, and L. Fei-Fei, "Rekep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation," in *CoRL*. PMLR, 2025.

[33] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," in *Robotics: Science and Systems XIX, Daegu, Republic of Korea, July 10-14, 2023*, 2023.

[34] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel *et al.*, "Diffusion policy: Visuomotor policy learning via action diffusion," *The International Journal of Robotics Research*, vol. 44, no. 10-11, pp. 1684–1704, 2025.

[35] A. Wang, L. Liu, H. Chen, Z. Lin, J. Han, and G. Ding, "Yoloe: Real-time seeing anything," in *ICCV*, October 2025, pp. 24 591–24 602.

[36] J. Yang, H. Zhang, F. Li, X. Zou, C. Li, and J. Gao, "Set-of-mark prompting unleashes extraordinary visual grounding in gpt-4v," *arXiv preprint arXiv:2310.11441*, 2023.

[37] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal *et al.*, "Learning transferable visual models from natural language supervision," in *ICML*. PMLR, 2021.

[38] H. K. Cheng, S. W. Oh, B. Price, J.-Y. Lee, and A. Schwing, "Putting the object back into video object segmentation," in *CVPR*, 2024, pp. 3151–3161.

[39] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay *et al.*, "Progprompt: Generating situated robot task plans using large language models," in *ICRA*, 2023, pp. 11 523–11 530.

[40] A. J. Hancock, A. Z. Ren, and A. Majumdar, "Run-time observation interventions make vision-language-action models more visually robust," in *ICRA*, 2025, pp. 9499–9506.

[41] E. J. Hu, yelong shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang *et al.*, "LoRA: Low-rank adaptation of large language models," in *ICLR*, 2022.

[42] OpenAI, "Gpt-4 technical report," 2024.